

Python for Professionals

A practical reference for building typed, tested FastAPI services with Python, SQLAlchemy, authentication, and production quality checks.

This handbook summarizes the course at python-pros.netlify.app, including the introduction, chapters 1 through 12, and the exercises. It is intended as a quick reference; consult the original course for detailed explanations and complete examples.

1. Professional Python Foundations

- Use uv for project creation, dependency management, lockfiles, and repeatable commands through uv run.
- Prefer the standard library, descriptive names, small functions, explicit return types, and keyword-only arguments when they clarify intent.
- Use enums for finite domain values, dataclasses or focused classes for structured state, and comprehensions only when they remain readable.
- Use context managers for resources and decorators for well-defined cross-cutting behavior. Avoid bare except blocks and silently ignored failures.
- Read tracebacks from the final exception back through project frames, then reproduce the smallest failing case.

Examples

```
from dataclasses import dataclass

@dataclass(frozen=True)
class Project:
    name: str
    is_active: bool = True

from contextlib import closing

with closing(resource) as managed_resource:
    managed_resource.process()

def total_price(*, price: float, tax_rate: float = 0.0) -> float:
    return price * (1 + tax_rate)
```

2. Types, Validation, and Tests

- Type public APIs, models, settings, and non-obvious values. Treat type errors as design feedback rather than suppressing them broadly.
- Use Pydantic at external boundaries to validate data and make accepted input explicit. Keep validation separate from business logic.
- Use pytest for focused behavior tests, fixtures for repeatable setup, parametrization for input matrices, and regression tests for fixed bugs.
- Test success and failure paths: invalid input, missing records, authorization failures, and database errors.
- Coverage identifies untested areas; it does not prove that tests assert meaningful behavior. Branch coverage helps expose one-sided conditionals.

Examples

```

from pydantic import BaseModel, Field

class ProjectInput(BaseModel):
    name: str = Field(min_length=1, max_length=100)

def display_name(name: str | None) -> str:
    return name.strip() if name else "Unnamed"

def test_total_price():
    assert total_price(price=10, tax_rate=0.2) == 12

```

3. FastAPI Application Design

- Keep route handlers thin: validate input, call application or data-layer code, and translate results into deliberate HTTP responses.
- Use typed Pydantic request and response models instead of unstructured dictionaries at API boundaries.
- Use `async def` for non-blocking async I/O and synchronous handlers for synchronous libraries; do not add `async` without an `async` dependency chain.
- Declare path, query, and body parameters explicitly. Use `APIRouter` to group routes and keep the application entry point small.
- Use Annotated dependency aliases for database sessions, authenticated users, and reusable request concerns.
- Centralize domain exception handling, document responses through OpenAPI metadata, and provide a safe health endpoint.

Examples

```

from fastapi import FastAPI

app = FastAPI()

@app.get("/health")
def health() -> dict[str, str]:
    return {"status": "ok"}

from pydantic import BaseModel

class ProjectResponse(BaseModel):
    id: int
    name: str

@router.get("/projects/{project_id}", response_model=ProjectResponse)
def get_project(project_id: int):
    return service.get_project(project_id)

```

4. SQLAlchemy, PostgreSQL, and Alembic

- Model relationships, foreign keys, indexes, nullability, and constraints explicitly. Use one session per request or operation.
- Perform filtering and relational work in the database, handle missing records deliberately, and inspect relationship loading for N+1 queries.
- Use Alembic migrations as permanent version-controlled history. Autogenerate is a draft, not an approval: review every operation and downgrade.
- Before applying a migration, verify that models are visible to SQLAlchemy metadata and inspect the generated foreign keys, indexes, enums, and nullability.

- Use alembic current to inspect state, alembic upgrade head to apply pending revisions, and alembic downgrade -1 for a controlled rollback.
- Never edit an applied migration in a shared environment; create a new migration on top.

Examples

```
from sqlmodel import Field, SQLModel

class Project(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)

statement = select(Project).where(Project.name == name)
project = session.exec(statement).first()
if project is None:
    raise ProjectNotFoundError(name)

uv run alembic revision --autogenerate -m "add projects table"
uv run alembic upgrade head
uv run alembic current
```

5. Dependencies, CRUD, and Middleware

- Use dependency injection to remove repeated session and lookup logic from route handlers.
- Separate create, read, update, and delete behavior, and test each operation at the API boundary.
- Use logging rather than print. Configure logging at the application boundary and use module-level loggers.
- Use middleware only for cross-cutting concerns such as timing, correlation IDs, or safe request logging; keep business logic out of it.

Examples

```
from typing import Annotated
from fastapi import Depends

ProjectDep = Annotated[Project, Depends(get_project)]

def read_project(project: ProjectDep):
    return project

import logging

logger = logging.getLogger(__name__)
logger.info("project_created", extra={"project_id": project.id})

@app.middleware("http")
async def add_request_id(request, call_next):
    response = await call_next(request)
    response.headers["X-Request-ID"] = request_id
    return response
```

6. Authentication and Security

- Hash passwords with a modern password-hashing library; never store plaintext passwords.
- Use signed, expiring JWTs only when stateless authentication fits the application. Validate signature, algorithm, and expiration.

- Store signing keys in environment-backed settings using SecretStr or an equivalent secret-aware type. Require a strong key and fail fast when it is missing.
- Put a stable user identity in the subject claim and protect routes through a reusable authentication dependency.
- Return generic authentication failures and never log passwords, tokens, signing keys, database credentials, or authorization headers.

Examples

```
from pwdlib import PasswordHash

password_hash = PasswordHash.recommended()
stored = password_hash.hash(plain_password)
password_hash.verify(plain_password, stored)

from datetime import UTC, datetime, timedelta

payload = {"sub": user_id, "exp": datetime.now(UTC) + timedelta(minutes=60)}
token = jwt.encode(payload, settings.jwt_secret_key.get_secret_value(), algorithm="HS256")

def require_user(token: Annotated[str, Depends(oauth2_scheme)]) -> User:
    payload = jwt.decode(token, secret, algorithms=["HS256"])
    return user_repository.get_required(payload["sub"])
```

7. Quality Checks and Delivery

- Keep linting, formatting, type checking, testing, and coverage commands identical locally and in CI.
- Use Docker for reproducible service environments. Keep production images focused on runtime dependencies and never copy secrets into them.
- Use environment or platform secret management for deployed credentials. Keep .env files and workflow secrets out of version control.
- CI should verify formatting and linting rather than silently rewriting code.

Examples

```
uv add --dev coverage mypy pytest ruff
uv run ruff check --no-cache .
uv run ruff format --check .

uv run mypy .
uv run coverage run -m pytest
uv run coverage report

FROM python:3.14-slim
WORKDIR /app
COPY pyproject.toml uv.lock ./
RUN pip install uv && uv sync --frozen --no-dev
COPY src ./src
```

Recommended command sequence

```
uv run ruff check --no-cache .
uv run ruff format --check .
uv run mypy .
uv run coverage run -m pytest
uv run coverage report
```

Course Roadmap

Chapter	Focus
Introduction	uv, environment setup, VS Code, running code, and tracebacks
1-2	Python syntax, data types, functions, classes, collections, exceptions, context managers, and decorators
3	Type hints, static checking, Pydantic, and pytest
4	FastAPI routes, async versus sync, JSON, OpenAPI, and endpoint tests
5-6	PostgreSQL, SQLAlchemy, Alembic, database-backed reads, writes, and CRUD refactoring
7-8	Dependencies, exceptions, APIRouter, logging, middleware, and schema evolution
9	Task endpoints, relational queries, filtering, and database fixtures
10	Password hashing, JWTs, and protected endpoints
11	Ruff, mypy, pytest, coverage, Docker, secrets, and GitHub Actions
12	Release Tracker walkthrough, frontend connection, wrap-up, and resources

Debugging loop

- Read the entire traceback and identify the first project frame that violates an assumption.
- Reproduce the smallest failing case and inspect values and types at the boundary.
- Use the VS Code debugger for request flows that logs cannot make clear.
- Add a regression test before broadening the fix.

Source: Python for Professionals, <https://python-pros.netlify.app/>. Prepared September 23, 2026.